nutrikit · Estado del proyecto

Qué existe hoy, qué quedó pendiente y qué issues siguen

Fecha: **27 de octubre de 2026** Rama: **main** (`c060a0c`) Fase: **0 (fundamentos) terminada**

10/11 issues de la Fase 0 cerrados (#11 es la guía)	**139** pruebas en verde, con Postgres real	**5** migraciones de Flyway (V1–V5)	**6** decisiones de arquitectura (ADR)

Este documento resume lo que ya está construido en el repositorio, lo que quedó pendiente o a medias, y una propuesta de issues para la Fase 0 restante y las fases 1 a 4. Sirve como insumo para escribir los siguientes issues; cada propuesta indica de qué depende y en qué fase entra según el modelo de datos.

1. Lo que ya existe

1.1 Repositorio y forma de trabajo

Pieza	Estado
Monorepo	`backend/` (Spring Boot), `frontend/` y `infra/`. `frontend/` solo tiene un README.
Flujo de trabajo	`CONTRIBUTING.md` con GitHub Flow, ramas `tipo/numero-descripcion`, Conventional Commits y la definición de terminado. Plantilla de PR en `.github/pull_request_template.md`.
Integración continua	Jenkins local en Docker (`infra/jenkins/`) con Configuration as Code, plugins fijos, JDK 25 y un Multibranch Pipeline que construye ramas y PRs. El `Jenkinsfile` corre `./mvnw clean verify`, publica JUnit y archiva los diagramas de Spring Modulith. Se probó en verde (139 pruebas) y en rojo (prueba fallida inyectada).
Desarrollo local	`./mvnw spring-boot:run` activa el perfil `dev`: levanta Postgres 17 con Docker Compose (puerto 5433), clave JWT efímera, operador local y Swagger UI.
Documentación	README con stack, arquitectura y cómo probar la API con `curl`. Diagramas C4 nivel 1 y 2 en Mermaid (`docs/architecture/`). Modelo de datos en PDF. Seis ADR.

1.2 Backend: stack

Java 25 · Spring Boot 4.1 · Spring Modulith 2.1 · Spring Security 7 (resource server JWT) · Spring Data JPA con Hibernate 7.4 · PostgreSQL 17 · Flyway · springdoc 3.2 · JUnit 5, Testcontainers y Scenario de Modulith.

1.3 Módulos (Spring Modulith)

Un paquete por módulo bajo `dev.jorgemiguel.nutrikit`. La raíz de cada paquete es su API pública; los subpaquetes son internos. `ModularityTests` rompe el build si un módulo usa clases internas de otro o si hay ciclos.

Módulo	Fase	Estado	Qué tiene
shared	—	Hecho	`TenantContext` (ScopedValue), filtro de Hibernate por `tenant_id`, `JpaTransactionManager` que fija `app.tenant_id` por transacción, formato de errores Problem Details, OpenAPI, `Origen` de cada acción, reintento de eventos pendientes y reloj.
tenants	F0	Hecho	Entidad `Tenant` con slug como value object y ciclo de vida (ACTIVO ⇄ SUSPENDIDO → CANCELADO). API de operador. Resolución del consultorio por subdominio o cabecera `X-Tenant` (dev). Módulos por consultorio (`TenantModulo`) solo de lectura. API pública: `Consultorios` y los eventos `TenantCreado` y `EstadoDeTenantCambiado`.
iam	F0	Hecho	Usuarios por consultorio (TITULAR, COLABORADOR, ASISTENTE), login con JWT HS256 de 15 min, refresh tokens rotativos con detección de reutilización, límite de intentos (429), operador por variables de entorno, alta de usuarios por el operador. Eventos de inicio de sesión exitoso y fallido.
audit	F0	Hecho	Bitácora append-only alimentada por eventos (después del commit, asíncrona, con outbox y reintentos). Tabla particionada por mes.
patients	F1	Vacío	Solo `package-info.java`.
patientaccess	F1	Vacío	Solo `package-info.java`. Decisión tomada en ADR-0003.
consultations	F1	Vacío	Solo `package-info.java`.
mealplans, integrations	F2	Vacío	Solo `package-info.java`.
scheduling, website	F3	Vacío	Solo `package-info.java`.

1.4 Base de datos

Migración	Contenido
`V1__tenants`	`tenant` y `tenant_modulo`.
`V2__event_publication`	Registro de eventos de Spring Modulith (outbox), estructura v2.
`V3__permisos_app_y_rls`	Permisos del usuario de la app y RLS en `tenant_modulo`.
`V4__iam`	`usuario` (correo `citext` único por consultorio) y `refresh_token` (solo hash SHA-256), ambas con RLS.
`V5__auditoria`	`auditoria` particionada por mes, solo `INSERT` y `SELECT` para la app, RLS y la función `asegurar_particion_auditoria` (`SECURITY DEFINER`).

- **Dos usuarios:** Flyway migra como dueño del esquema; la app se conecta como `nutrikit_app`, sin DDL, sin ser dueña de las tablas y sin `BYPASSRLS`.
- **Aislamiento entre consultorios en dos barreras:** filtro de Hibernate siempre activo y Row-Level Security con `app.tenant_id` local a cada transacción. La tabla `tenant` no lleva RLS.
- **Convenciones aplicadas:** UUIDv7 como llave primaria (salvo `auditoria`, que usa bigint), `timestamptz`, enums como `varchar` con `CHECK`, `ddl-auto=validate`.

1.5 API

Ruta	Acceso	Qué hace
`POST /api/operador/auth/login`	Público	Access token del operador (sin refresh).
`POST /api/operador/tenants`	OPERADOR	Alta de consultorio: 201 con `Location`, 409 si el slug existe.
`GET /api/operador/tenants[/{id}]`	OPERADOR	Listado paginado (máx. 100) y detalle.
`PATCH /api/operador/tenants/{id}/estado`	OPERADOR	Cambio de estado; 409 si la transición no se permite.
`POST /api/operador/tenants/{id}/usuarios`	OPERADOR	Alta de usuario de un consultorio.
`POST /api/auth/login | refresh | logout`	Público, en el consultorio	Sesión de los usuarios del consultorio.
`GET /api/consultorio`, `/api/consultorio/modulos`	JWT del mismo consultorio	Datos y módulos del consultorio actual.

Todos los errores siguen Problem Details (RFC 9457), incluidos 401 y 403 de Spring Security. Swagger UI solo en el perfil `dev`.

1.6 Pruebas

Área	Qué cubre
Dominio (sin Spring)	`SlugTest`, `TenantTest`, `ResolutorDeTenantTest`, `LimitadorDeIntentosTest`, `JwtConfigTest`.
Persistencia y aislamiento	`TenantRepositoryTest`; `AislamientoTenantTest` prueba cada barrera por separado, la escritura cruzada y los permisos del usuario de la app.
API y seguridad	`TenantControllerTest`, `TenantApiTest`, `ResolucionTenantTest`, `AutenticacionTest` (26 pruebas con tokens reales).
Auditoría y eventos	`AuditoriaTest` (Scenario por evento, flujo HTTP, append-only, particiones) y `ReintentoDeAuditoriaTest`.
Arquitectura	`ModularityTests`: límites entre módulos, `audit` solo por eventos y generación de diagramas.

1.7 Decisiones de arquitectura (ADR)

#	Decisión
0001	Monolito modular con Spring Modulith.
0002	Multi-tenant con base compartida, `tenant_id` y RLS.
0003	Acceso del paciente por enlace privado, con PIN opcional (aún sin implementar).
0004	Implementación del aislamiento: identificación, ScopedValue, filtro de Hibernate, RLS por transacción y dos usuarios de base.
0005	Autenticación con JWT propios y refresh tokens rotativos.

#	Decisión
0006	Bitácora de auditoría con eventos de dominio.

2. Pendientes y deuda conocida

Salen de los ADR, de criterios que quedaron a medias y de diferencias entre el modelo de datos y lo implementado. Prioridad: **Alta** bloquea algo o es un riesgo; **Media** conviene antes de la Fase 1; **Baja** puede esperar.

2.1 Proceso y repositorio

Prioridad	Pendiente	Detalle
Alta	`main` sin protección	No hay ruleset ni protección de rama, y siguen habilitados merge commit, squash y rebase. Los PR se integraron con merge commit. Faltan los criterios de #2 “push directo rechazado” y “una sola estrategia de merge”.
Media	`CLAUDE.md` (issue #4)	El issue no aparece en GitHub y el archivo no existe.
Media	Jenkins no reporta a GitHub	El resultado del build no aparece como check en los PR, así que no se puede exigir “build en verde” para integrar. Tampoco hay token de GitHub configurado (límite de 60 peticiones por hora).
Baja	Calidad en el pipeline	Sin cobertura (JaCoCo), análisis estático (SpotBugs, Sonar) ni revisión de dependencias vulnerables.
Baja	Imagen Docker del backend	No hay `Dockerfile` de la app; el C4 nivel 2 indica que Jenkins despliega una imagen.

2.2 Modelo de datos

Prioridad	Pendiente	Detalle
Media	PDF del modelo desactualizado	Falta `tenant_id` en `refresh_token`; faltan los valores de `auditoria.actor_tipo` (USUARIO, OPERADOR, ANONIMO, SISTEMA); el operador no está en el modelo.
Media	Migraciones por módulo	El modelo pide una carpeta de Flyway por módulo (`db/migration/pacientes` …); hoy todas van en una sola. Conviene decidirlo antes de que haya más módulos.
Baja	`tenant_modulo` incompleto	`modulo` es `varchar` sin `CHECK` (el modelo dice enum) y `config jsonb` no está mapeado.

2.3 Funcionalidad de Fase 0 a medias

Prioridad	Pendiente	Detalle
Alta	Encender y apagar módulos	El operador no tiene endpoint para activar módulos, y un módulo apagado sigue exponiendo sus endpoints. Es la base del modelo “núcleo + módulos por consultorio”.
Media	Gestión de usuarios	La titular no puede listar, invitar, desactivar ni cambiar el rol de su equipo. No hay cambio ni recuperación de contraseña. Los roles todavía no restringen nada dentro del consultorio.

Prioridad	Pendiente	Detalle
Media	Limpieza de refresh tokens	Los tokens vencidos y revocados se acumulan (ADR-0005).
Baja	Auditoría de plataforma	El login y las acciones del operador fuera de un consultorio no se auditan (ADR-0006).
Baja	Retención de la bitácora	Falta definir cuánto tiempo se conservan las particiones (NOM-004: al menos 5 años).

2.4 Antes de producción

Prioridad	Pendiente	Detalle
Media	IP real detrás de un proxy	Configurar `server.forward-headers-strategy`; si no, el límite de intentos y la auditoría ven la IP del balanceador.
Media	Límite de intentos compartido	Vive en memoria; con varias réplicas hay que moverlo a Redis (Bucket4j).
Media	CORS	No está configurado; hace falta en cuanto exista el frontend en otro origen.
Baja	Observabilidad	Actuator solo expone `health`, `info` y `modulith`; faltan métricas, trazas y logs estructurados.
Baja	Caché de resolución del consultorio	Cada petición consulta `tenant` (ADR-0004).

3. Propuesta de issues

Ordenados para que cada uno se apoye en los anteriores. Los títulos son sugerencias; cada uno debería escribirse con el formato de la Fase 0 (objetivo, contexto, qué estudiar, qué hacer, criterios de aceptación y preguntas para reflexionar).

3.1 Cierre de la Fase 0

#	Issue sugerido	Depende de	Alcance
A1	Proteger `main` y exigir el build de Jenkins	—	Ruleset sin push directo, solo squash merge, borrar ramas al integrar. Jenkins publica el estado del commit en GitHub y el ruleset lo exige.
A2	`CLAUDE.md` del repositorio	—	Recrear el issue #4 con lo aprendido: dos barreras de aislamiento, dos usuarios de base, eventos y reglas de migraciones.
A3	Encender y apagar módulos por consultorio	—	Endpoint de operador para habilitar módulos y su configuración; `CHECK` de módulos válidos; un módulo apagado responde 404 en sus rutas; evento `ModuloCambiado` auditado.
A4	Gestión del equipo del consultorio	A3	La titular lista, crea, desactiva y cambia el rol de sus usuarios; autorización por rol con `@PreAuthorize`; desactivar revoca sus refresh tokens.
A5	Cambio y recuperación de contraseña	A4	Cambio con la contraseña actual; recuperación con token de un solo uso por correo (SES simulado en dev); revoca las sesiones; auditado.

#	Issue sugerido	Depende de	Alcance
A6	Migraciones de Flyway por módulo	—	Decidir y aplicar una carpeta por módulo, con su ADR, sin romper las bases existentes.
A7	Calidad en el pipeline	A1	Cobertura con JaCoCo publicada en Jenkins, análisis estático y revisión de dependencias con vulnerabilidades conocidas.
A8	Mantenimiento: limpieza de tokens y actualizar el modelo de datos	—	Tarea programada que borra refresh tokens vencidos; actualizar el PDF del modelo (`refresh_token.tenant_id`, `actor_tipo`, operador).

3.2 Fase 1: expediente y acceso del paciente

#	Issue sugerido	Depende de	Alcance
B1	Pacientes: alta, edición, baja lógica y búsqueda	A3	Tabla `paciente` con RLS, baja lógica con `eliminado_en`, búsqueda por nombre con índice trigram, paginación; eventos auditados.
B2	Cifrado de campos sensibles	B1	AES-256-GCM a nivel de aplicación con una llave por consultorio (llave maestra local en dev, KMS después); ADR.
B3	Historia clínica cifrada	B2	`historia_clinica` con columnas `ENC`, una por paciente; cada lectura queda en la bitácora (NOM-004).
B4	Consentimiento del aviso de privacidad	B1	Registrar versión, fecha y medio; sin consentimiento no se guardan datos de salud.
B5	Catálogo de indicadores	A3	Indicadores globales (peso, IMC…) y propios del consultorio; los calculados con fórmula.
B6	Consultas y mediciones	B1, B5	Consulta con notas cifradas y mediciones por indicador (`UNIQUE (consulta_id, indicador_id)`); serie histórica por indicador para gráficas.
B7	Enlace privado del paciente	B1	ADR-0003: token de 128 bits guardado como hash, PIN con Argon2, caducidad, revocación, rate limiting, `noindex`; cada acceso auditado.
B8	Página privada del paciente (API)	B6, B7	Solo lectura de su progreso, limitada a los indicadores que la nutrióloga decide mostrar.
B9	Frontend base	A4	React + TypeScript + Vite: login por subdominio, manejo de tokens, panel con los módulos encendidos, CORS en el backend, paso de build en Jenkins.
B10	Pantallas de pacientes y consultas	B6, B9	Expediente, captura de consulta y gráficas de progreso.

3.3 Fases 2 a 4

#	Issue sugerido	Fase	Alcance
C1	Adjuntos cifrados en S3	F2	URLs prefirmadas, cifrado en reposo, límites de tamaño y tipo; MinIO o LocalStack en desarrollo.

#	Issue sugerido	Fase	Alcance
C2	Importación de mediciones de InBody	F2	Puerto `ImportadorMediciones`, CSV de LookinBody, reporte de filas con error. Pendiente: conseguir un CSV real.
C3	Catálogo SMAE y planes de alimentación	F2	Catálogo global de equivalentes, plantillas, plan por paciente con un solo ACTIVO (índice parcial), menús y PDF del plan.
C4	Visibilidad de indicadores y branding	F2	`visibilidad_indicador` y `tenant_branding` (logo, colores, encabezado de PDF).
D1	Agenda: horarios, excepciones y citas	F3	Citas sin traslapes con `EXCLUDE USING gist`; zona horaria del consultorio en la presentación.
D2	Recordatorios por correo	F3	Amazon SES con un adaptador simulado en dev; programación y reintentos.
D3	Sitio público del consultorio	F3	Plantillas, versiones (BORRADOR, PUBLICADA, HISTÓRICA), publicación a S3 + CloudFront.
E1	Infraestructura en AWS con Terraform	F4	RDS, S3, CloudFront, SES y cómputo para el backend; secretos fuera del repo; despliegue desde Jenkins.
E2	Producción: proxy, límites compartidos, observabilidad	F4	IP real, límite de intentos en Redis, métricas y trazas, caché de consultorios, retención de la bitácora.
E3	Suscripciones	F4	Tabla `suscripcion` y su efecto sobre el estado del consultorio.

> **Por validar antes de escribir los issues de Fase 1 y 2** (señalado en el modelo de datos): campos exactos de la historia clínica y de los indicadores con nutriólogas reales, y el formato real del CSV de LookinBody.

Generado a partir del repositorio `MiguelLescasJorgeSebastian/nutrikit` (rama `main`, commit `c060a0c`), los issues y PR de GitHub, los ADR y `docs/modelo-de-datos.pdf` v0.1.